## 1. History of JAVA.

- Java was initially developed in 1991 named as "oak" but was renamed "Java" in 1995.
- Originally designed for small, embedded systems in electronic appliances like set-top boxes.
- The primary motivation was the need for a platform-independent language that could be used to create software to be embedded in various consumer electronic devices.
- Java programming language was originally developed by Sun Microsystems which was initiated by James Gosling and released in 1995 as core component of Sun Microsystems' Java platform (Java 1.0 [J2SE]).
- It promised Write Once, Run Anywhere (WORA), providing no-cost run-times on popular platforms.

## 2. What is Java?

Java is a programming language that:

- Is exclusively object oriented
- Has full GUI support
- Has full network support
- Is platform independent

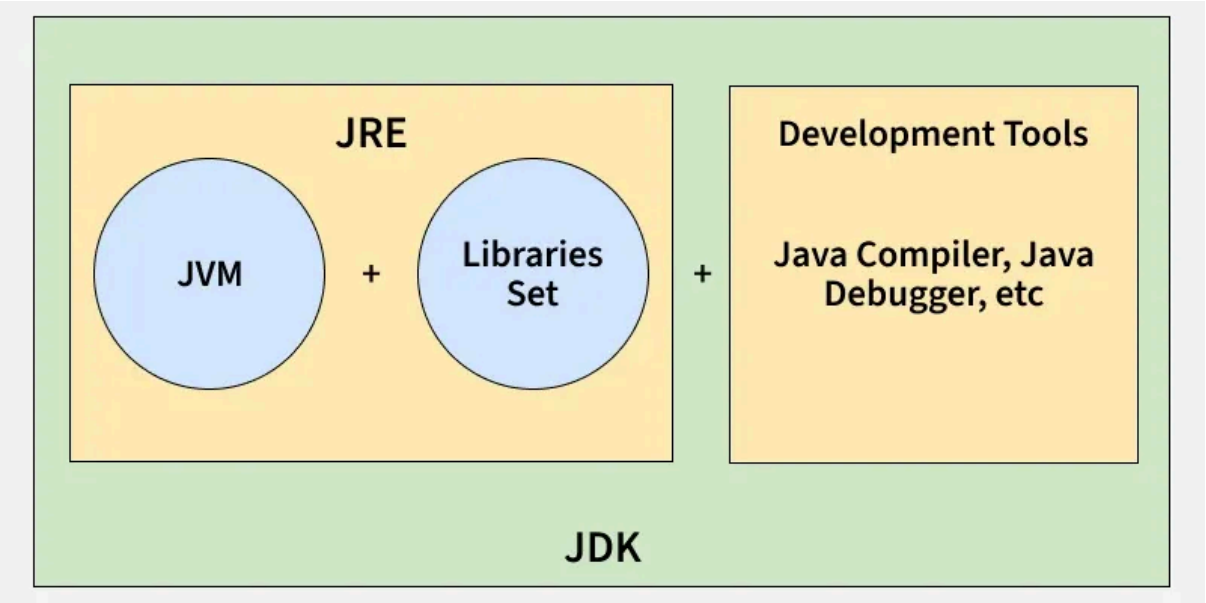
Application

- Desktop Applications such as acrobat reader, media player, antivirus, etc.
- Web Applications such as irctc.co.in, javatpoint.com, etc.
- Enterprise Applications such as banking applications.
- Mobile
- Embedded System
- Smart Card
- Robotics
- Games, etc.

## 3. Explain JVM, JRE, JDK and JIT.

**Key Notes:**

- **JVM** (Java Virtual Machine): Executes the Java bytecode.
- **JRE** (Java Runtime Environment): Provides the environment to run Java programs (includes JVM + libraries).
- **JDK** (Java Development Kit): Provides tools to develop and run Java programs (includes JRE + development tools).




## JDK (Java Development Kit)

- JDK is an acronym for **Java Development Kit.**
- The Java Development Kit (JDK) is a software development environment which is used to develop Java applications and applets.
- It **physically exists.**
- It contains JRE + development tools.
- When you download JDK, JRE is also downloaded with it.
- JDK is a platform-specific software and that's why we have separate installers for Windows, Mac, and Unix systems.

JDK is a software development kit used to build Java applications. It contains the JRE and a set of development tools.

- Includes compiler (javac), debugger, and utilities like jar and javadoc.
- Provides the JRE, so it also allows running Java programs.
- Required by developers to write, compile, and debug code.

Components of JDK:

- JRE (JVM + libraries)
- Development tools (compiler, jar, javadoc, debugger)

Note:

- JDK is only for development (it is not needed for running Java programs)
- JDK is platform-dependent (different version for windows, Linux, macOS)

Working of JDK:

- Source Code (.java): Developer writes a Java program.
- Compilation: The JDK's compiler (javac) converts the code into bytecode stored in .class files.
- Execution: The JVM executes the bytecode, translating it into native instructions.

## JRE(Java Runtime Environment)

- JRE is an acronym for Java Runtime Environment.
- JRE provides an environment to run Java programs but does not include development tools.
- It is also written as Java RTE.
- The Java Runtime Environment is a set of software tools which are used for developing Java applications.
- It is the implementation of JVM.
- It physically exists.
- The implementation of JVM is also actively released by other companies besides Sun Micro Systems.
- Contains the JVM and standard class libraries.
- Provides all runtime requirements for Java applications.
- Does not support compilation or debugging.

Note:

- JRE is only for running applications, not for developing them.
- It is platform-dependent (different builds for different OS).

Working of JRE:

1. Class Loading: Loads compiled .class files into memory.

2. Bytecode Verification: Ensures security and validity of bytecode.
3. Execution: Uses the JVM (interpreter + JIT compiler) to execute instructions and make system calls.

## JVM(Java Virtual Machine)

- JVM (Java Virtual Machine) is an abstract machine.
- It is called a virtual machine because it doesn't physically exist
- JVM is the core execution engine of Java.
- It is responsible for converting bytecode into machine-specific instructions.
  - Part of both JDK and JRE.
  - Performs memory management and garbage collection.
  - Provides portability by executing the same bytecode on different platforms.

**Note:**

- JVM implementations are platform-dependent.
- Bytecode is platform-independent and can run on any JVM.
- Modern JVMs rely heavily on Just-In-Time (JIT) compilation for performance.

**Working of JVM:**

- **Loading**: Class loader loads bytecode into memory.
- **Linking**: Performs verification, preparation, and resolution.
- **Initialization**: Executes class constructors and static initializers.
- **Execution**: Interprets or compiles bytecode into native code.

## Java In Time (JIT)

- JIT stands for **Just In Time compiler**
- Java was a interpreted programming language, but after introducing JIY it was called as interpreted-compiled programming language

- JIT increased the speed of execution
- JIT helps the JVM to find the active OS (Operating System)

### Advantages of JIT Compiler

- It requires less memory usages.
- The code optimization is done at run time.
- It uses different levels of optimization.
- It reduces the page faults.

### Disadvantages of JIT Compiler

- It increases the complexity of the program.
- The program with less line of code does not take the benefit of the JIT compilation.
- It uses lots of cache memory.

| Aspect | JDK (Java Development Kit) | JRE (Java Runtime Environment) | JVM (Java Virtual Machine) |
|---|---|---|---|
| Purpose | Used for Java development, including writing, compiling, and debugging code. | Used for executing Java applications. It contains the necessary runtime environment. | Executes Java bytecode on a specific platform. |
| Components | – Compiler (javac) for source code to bytecode conversion. – Debugger (jdb) for debugging. – Various tools and utilities. | – Java libraries and core classes. – JVM (Java Virtual Machine) for running Java bytecode. | – Responsible for executing Java bytecode. – Different implementations for various platforms. |

| | | | |
|---|---|---|---|
| Necessity for Running Java Code | Not required on the end-user's machine to run Java applications. | Required on the end-user's machine to run Java applications. | Required on the end-user's machine to execute Java applications. |
| Platform Independence | Code can be developed on one platform and executed on another with the appropriate JRE. | Provides the runtime environment needed to run Java applications on a specific platform. | Executes Java bytecode on the user's machine, making it platform-dependent. |
| Use in Development | Essential for writing and compiling Java code. | Not used during development but needed for running Java applications. | Not used directly in development, as it's responsible for executing compiled Java bytecode. |

## 4. List out features of Java

- Simple
- Secure
- Portable
- Object-oriented
- Robust
- Multithreaded
- Architecture-natural
- Interpreted
- High performance
- Distributed
- Dynamic

1. **Simple**

- It's simple because it contains many features of other languages like C and C++
- It also removed complexities like pointers, Storage classes, goto statement and multiple Inheritance.

2. **Secure**

- Java is best known for its security.
- With Java, we can develop virus-free systems.
- Java is secured because:

  No explicit pointer

  Java Programs run inside virtual machine sandbox
  Bytecode Verifier

3. **Portable**

- Java is portable because it facilitates you to carry the java bytecode to any platform.

4. **Object oriented**

- Java is Object-oriented programming language. Everything in Java is an object.

5. **Robust**

Robust simply means strong. Java is robust because:

- ☐ It uses strong memory management.
- ☐ There are lack of pointers that avoids security problem.
- ☐ There is automatic garbage collection in java.
- ☐ There is exception handling and type checking mechanism in java.

6. **Multithreaded**

- A thread is like a separate program, executing concurrently.
- We can write Java programs that deal with many tasks at once by defining multiple threads.
- The main advantage of multi-threading is that it doesn't occupy memory for each thread.
- It shares a common memory area. Threads are important for multi-media, Web applications etc…

7. **Architecture-neutral**

- Java is architecture neutral because there is no implementation dependent features

e.g. size of primitive types is fixed.

Example : in c int occupy 2 byte for 32 bit OS and 4 bytes for 64 bit OS whereas in

JAVA it occupy 4 byte for int both in 32 bit and 64 bit OS.

## 5. Java Program Structure, Compilation, and Execution

Java Program Structure

A Java program is made up of several components, primarily consisting of classes and methods. Here's a basic outline of the structure of a Java program:

```
// Import necessary classes (optional)

import java.util.Scanner;

// Define a class

public class MyProgram {

                    // Main method: entry point of the program

  public static void main(String[] args) {

                    // Inside the main method, you write the program logic

    System.out.println("Hello, World!");  // Prints message to the console

  }

}
```

**Key Components:**

- **Package Declaration** (optional): Sometimes a package declaration is included at the start, which helps organize Java files.
- **Imports (optional):** Import statements are used to include classes from Java libraries (e.g., import java.util.Scanner;).
- **Class Definition:** Every Java program must have at least one class. The class is defined using the class keyword followed by the class name (e.g., public class MyProgram).
- **Main Method:** The main method is the starting point for any Java application. It's defined as public static void main(String[] args).

Java is a high-level, platform-independent language. To run a Java program, it must go through compilation and execution. Understanding this process is essential for beginners. For example, a simple

Java program:

```
public class Geeks{

  public static void main(String[] args){

    System.out.println("Hello, Geeks");
  }
}
```

**Output**

Hello, Geeks

**How to run the above code?**

- Write code in a file like Geeks.java.
- The Java Compiler "javac" compiles it into bytecode "Geeks.class".
- The JVM (Java Virtual Machine) reads the .class file and interprets the bytecode.
- JVM converts bytecode to machine readable code i.e. "binary" (001001010) and then execute the program.

**Steps to Compile and Execute a Java Program**

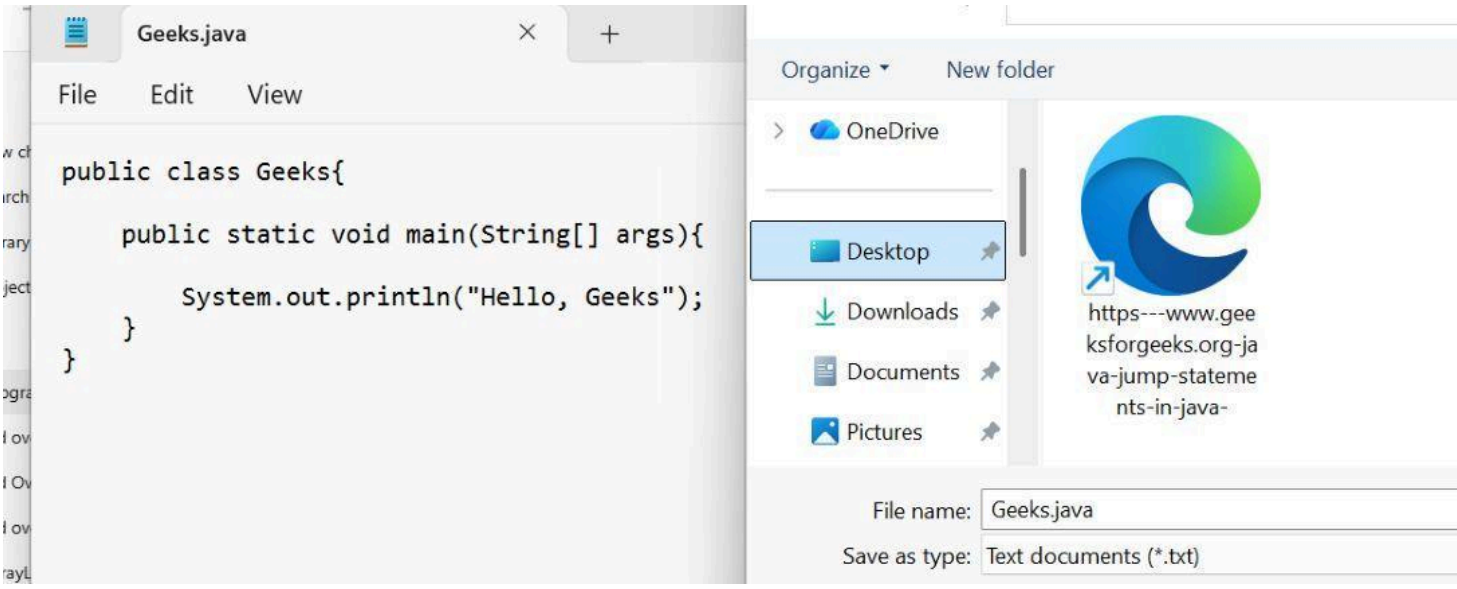
Running a Java program involves two main steps: compilation and execution. Let's look at each step in detail.

**Step 1: Writing the Java Program**

Before you can run a Java program, you need to write it in a text editor. You can use any text editor like Notepad (Windows), TextEdit (Mac), or an IDE (Integrated Development Environment) like Eclipse, IntelliJ IDEA, or VS Code.

**Example:**

- **Class name:** Geeks
- **File name:** Geeks.java




*output*

***Note:*** *Save the file with a .java extension and make sure the filename matches the class name.*

### Step 2: Compiling the Java Program

Once the Java program is written, it needs to be compiled. Compilation is the process of converting the human-readable Java code into bytecode, which can be understood by the Java Virtual Machine (JVM).

**Command to compile:**

*javac Geeks.java*

**Here:**

- **javac** is the Java compiler.
- **Geeks.java** is the file to compile.

*Output*

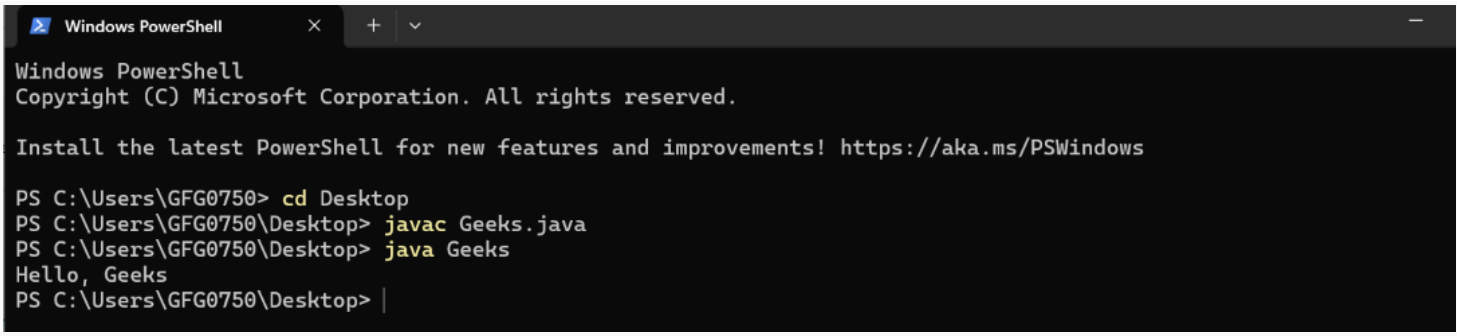
If there are no errors, this command will create a file called Geeks.class. The .class file contains bytecode, which is a platform-independent code that the JVM can execute. If there are errors, the compiler will show them in the terminal. You need to fix the errors before running the program.

**Step 3: Executing the Java Program**

After successful compilation, you can run the program using the Java interpreter.

**Command to run:**

*java Geeks*




*output*

**Here:**

- java is the Java interpreter.
- Geeks is the name of the class containing the main method (do not include .class).

**When executed, you will see:**

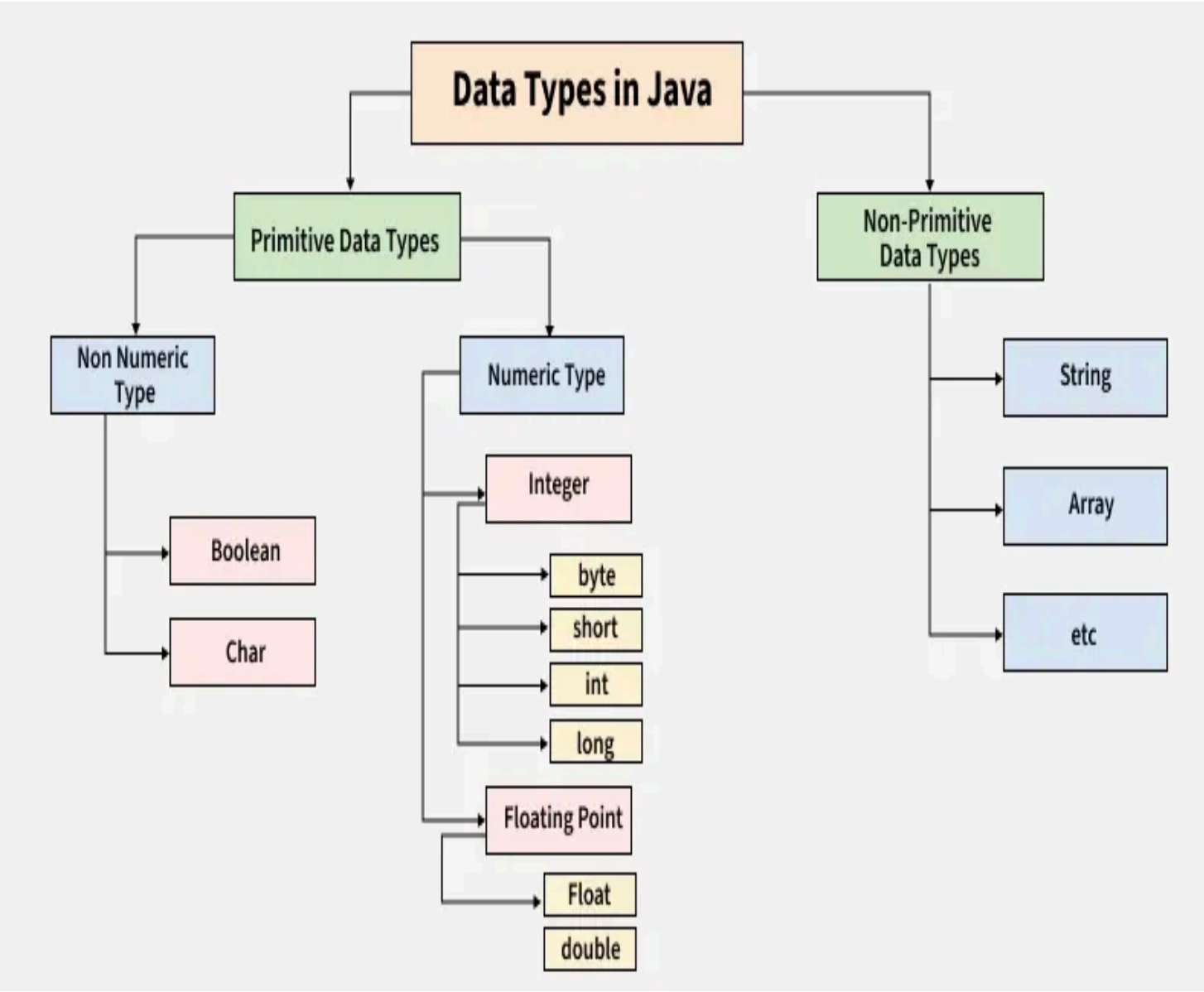
*Hello, Geeks*

## Data types (byte, short, int, long, float, double, char, boolean, String, Arrays)

## 6. Explain Data Types in detail with example.

Data types in Java define the kind of data a variable can hold and the memory required to store it. They are broadly divided into two categories:

- **Primitive Data Types:** Store simple values directly in memory.
- **Non-Primitive (Reference) Data Types:** Store memory references to objects.




## Primitive Data Types

A primitive data type specifies the type of a variable and the kind of values it can hold.

There are **eight** primitive data types in Java:

| Data Type | Size | Description |
|---|---|---|
| byte | 1 byte | Stores whole numbers from **-128 to 127** |
| short | 2 bytes | Stores whole numbers from **-32,768 to 32,767** |
| int | 4 bytes | Stores whole numbers from **-2,147,483,648 to 2,147,483,647 OR**<br><br>**$-2^{31}$ to $2^{31}-1$** |
| long | 8 bytes | Stores whole numbers from **-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807 OR**<br><br>**$-2^{63}$ to $2^{63}-1$** |
| float | 4 bytes | Stores **fractional** numbers. Sufficient for storing 6 to 7 decimal digits |
| double | 8 bytes | Stores fractional numbers. Sufficient for storing 15 to 16 decimal digits |
| boolean | 1 byte | Stores **true** or **false** values |
| char | 2 bytes | Stores a single **character**/letter or **ASCII** values |

## EXAMPLE:

```
int myNum = 5;                // Integer (whole number)

float myFloatNum = 5.99f;     // Floating point number
```

```
char myLetter = 'D';          // Character

boolean myBool = true;        // Boolean

String myText = "Hello";      // String
```

## Non-Primitive Data Types

The non-primitive data types are not predefined. The reference data types are created using defined constructors of the classes. They are used to access objects. These variables are declared to be of a specific type that cannot be changed. For example, Employee, Puppy, etc.

The following are the non-primitive (reference/object) data types –

**String**: The string is a class in Java, and it represents the sequences of characters.

**Arrays**: Arrays are created with the help of primitive data types and store multiple values of the same type.

**Classes**: The classes are the user-defined data types and consist of variables and methods.

**Interfaces**: The interfaces are abstract types that are used to specify a set of methods.

# ❖ type conversion

## 7. What is Type Casting? Explain widening and narrowing type casting.

- Type casting is the process of converting a value from one data type to another.
- Converts a **smaller data type to a larger data type**
- Done **automatically** by Java
- No data loss

**KEY NOTES:**

**Widening Casting (automatic/Implicit)** - converting a **smaller type to a larger type** size

`byte` -> `short` -> `char` -> `int` -> `long` -> `float` -> `double`

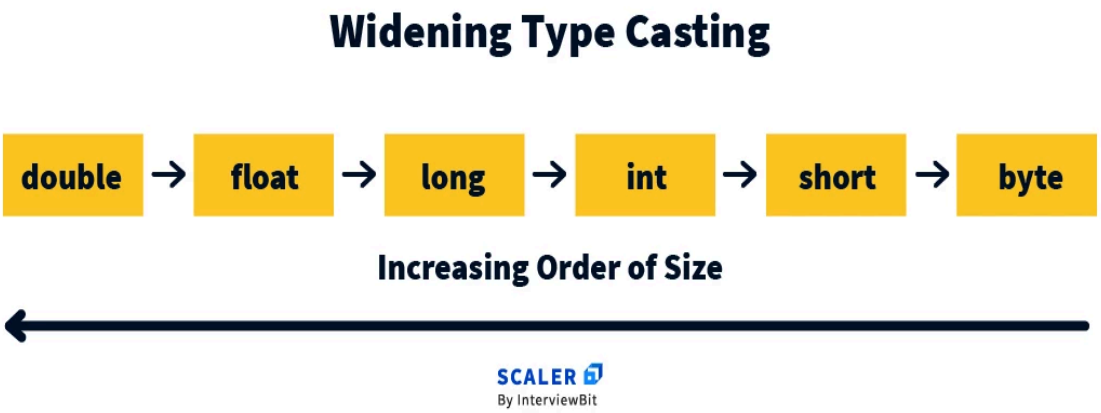
**Narrowing Casting (manual)** - converting a **larger type to a smaller type** size

`double` -> `float` -> `long` -> `int` -> `char` -> `short` -> `byte`

There are two primary types of casting: widening and narrowing.

## 1. Widening Type Casting (Implicit Casting)

Widening casting, also known as **implicit** casting, occurs when converting a value of a smaller data type to a larger data type.




**Syntax:**

*larger_data_type variable_name = smaller_data_type_variable;*

**Examples:**

- Converting an `int` to a `long`.
- Converting a `float` to a `double`.
- Converting a `char` to an `int`

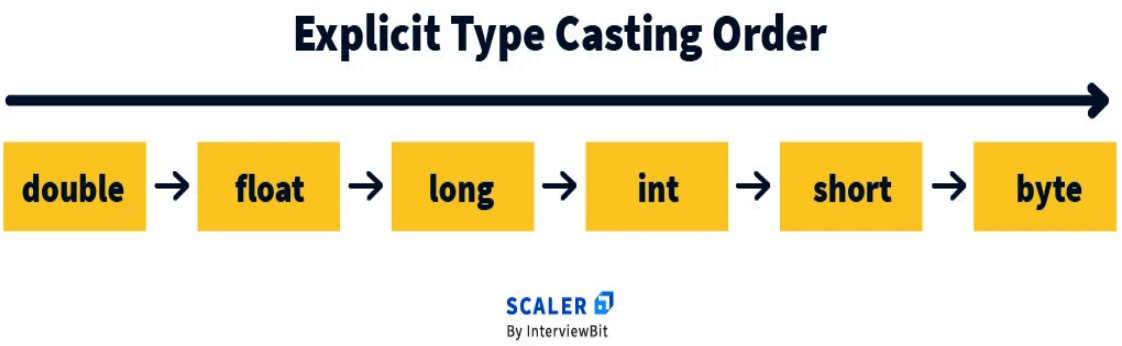
# Example

```
int myInt = 9;
```

```
double myDouble = myInt; // Automatic casting: int to double
```

## 2. Narrowing Type Casting (Explicit Casting)

- Narrowing casting, also known as **explicit** casting, occurs when converting a value of a larger data type to a smaller data type.
- Converts a **larger data type to a smaller data type**
- Must be done **manually**
- May cause **data loss**




**Syntax:**

***smaller_data_type variable_name = (smaller_data_type) larger_data_type_variable;***

**Examples:**

- Converting a `double` to an `int`. The fractional part will be truncated.
- Converting a `long` to an `short`. The value might overflow or be clipped.
- Converting an `int` to a `chaR`.

**Example:**

double a = 10.5;

int b = (int) a;  // narrowing

✔ Explicit cast required

❌ Decimal part lost (10.5 → 10)

## ❖ variables, keywords, literal, operators, operator precedence and associativity

### 1. Variables

In Java, variables are containers used to store data in memory. Variables define how data is stored, accessed, and manipulated.

A variable in Java has three components,

- Data Type: Defines the kind of data stored (e.g., int, String, float).
- Variable Name: A unique identifier following Java naming rules.
- Value: The actual data assigned to the variable.

```
class Geeks {
   public static void main(String[] args)
       // Declaring and initializing variable
       // Integer variable
       int age = 25;
       // String variable
       String name = "GeeksforGeeks"
       // Double variable
       double salary = 50000.50;
       // Displaying the values of variables
       System.out.println("Age: " + age);
       System.out.println("Name: " + name);
       System.out.println("Salary: " + salary);
   }
}
```

Output

```
Age: 25
```

```
Name: GeeksforGeeks
```

```
Salary: 50000.5
```

## How to Initialize Java Variables?

### Example: Here, we are initializing variables of different types like float, int and char.

```
class Geeks{
   public static void main(String[] args) {
       // Declaring and initializing variables

       // Initializing float variable
       float si = 5.5f;

       // Initializing integer variables
       int t = 10;
       int s = 20;

       // Initializing character variable
       char var = 'h';

       // Displaying the values of the variables
       System.out.println("Simple Interest: " + si);
       System.out.println("Speed: " + s);
       System.out.println("Time: " + t);
       System.out.println("Character: " + var);
   }
}
```

### Output

```
Simple Interest: 5.5
```

```
Speed: 20
```

```
Time: 10
```

```
Character: h
```

## Rules to Name Java Variables

- Start with a Letter, $, or _ – Variable names must begin with a letter (a–z, A–Z), dollar sign $, or underscore _.
- No Keywords: Reserved Java keywords (e.g., int, class, if) cannot be used as variable names.
- Case Sensitive: age and Age are treated as different variables.
- Use Letters, Digits, $, or _ : After the first character, you can use letters, digits (0–9), $, or _.
- Meaningful Names: Choose descriptive names that reflect the purpose of the variable (e.g., studentName instead of s).
- No Spaces: Variable names cannot contain spaces.
- Follow Naming Conventions: Typically, use camelCase for variable names in Java (e.g., totalMarks).

## 2. Keywords

- In Java, keywords are the reserved words that have some predefined meanings and are used by the Java compiler for some internal process or represent some predefined actions.
- These words cannot be used as identifiers such as variable names, method names, class names, or object names.

**Used to define variable types and specify the kind of data they can hold.**

| Keyword | Usage |
|---|---|

| | |
|---|---|
| boolean | Defines a variable that holds true or false. |
| byte | Defines an 8-bit signed integer. |
| char | Defines a 16-bit Unicode character. |
| short | Defines a 16-bit signed integer. |
| int | Defines a 32-bit signed integer. |
| long | Defines a 64-bit signed integer. |
| float | Defines a 32-bit floating-point number. |
| double | Defines a 64-bit floating-point number. |

| | |
|---|---|
| void | Specifies that a method does not return any value. |

## 2. Control Flow Keywords

Used to control the execution flow of a program, including loops, branching, and jumps.

| Keyword | Usage |
|---|---|
| if | Executes code when a condition is true. |
| else | Defines an alternate block when an if condition is false. |
| switch | Selects one block of code among multiple options. |
| case | Defines an individual branch in a switch statement. |

| | |
|---|---|
| default | Defines the block executed if no case matches. |
| for | Starts a for loop. |
| while | Starts a while loop. |
| do | Starts a do-while loop. |
| break | Terminates the nearest loop or switch. |
| continue | Skips to the next iteration of a loop. |
| return | Exits from a method and optionally returns a value. |

## 3. Exception Handling Keywords

Used for handling and managing runtime errors in programs.

| Keyword | Usage |
|---|---|
| try | Defines a block of code to test for exceptions. |
| catch | Defines a block to handle exceptions thrown by try. |
| finally | Defines a block that always executes after try and catch. |
| throw | Used to manually throw an exception. |
| throws | Declares the exceptions a method can throw. |
| assert | Tests assumptions during program execution for debugging. |

## 4. Object-Oriented Keywords

Used to define classes, interfaces, and objects, as well as inheritance and encapsulation properties.

| Keyword | Usage |
|---|---|
| class | Declares a class. |
| interface | Declares an interface. |
| extends | Indicates inheritance from a superclass. |
| implements | Indicates that a class implements an interface. |
| new | Creates new objects. |
| this | Refers to the current object. |
| super | Refers to the superclass of the current object. |

| | |
|---|---|
| abstract | Declares a class or method that must be implemented in a subclass. |
| final | Prevents inheritance, overriding, or modification. |
| static | Declares class-level variables or methods. |
| sealed | Restricts which classes can extend a given class. |
| permits | Specifies the subclasses allowed to extend a sealed class. |

## 5. Access Control Keywords

Define the visibility or accessibility of classes, methods, and variables.

| Keyword | Usage |
|---|---|

| | |
|---|---|
| public | Accessible from anywhere in the program. |
| protected | Accessible within the same package or by subclasses. |
| private | Accessible only within the same class. |

### 6. Package and Import Keywords

Used to organize classes and access external code.

| Keyword | Usage |
|---|---|
| package | Defines a namespace for classes. |
| import | Allows access to classes from other packages. |

3. **Literals**

- In Java, literals are fixed, constant values that are directly represented in the source code.
- They are used to assign initial values to variables or to use directly in expressions, and their values do not change during program execution.

Java supports various types of literals:

- **Integer Literals:** Represent whole numbers and can be in decimal, octal (prefixed with `0`), hexadecimal (prefixed with `0x`), or binary (prefixed with `0b`) formats. Use an `L` or `l` suffix for `long` values.
- **Floating-Point Literals:** Represent numbers with fractional parts, defaulting to type `double`. They can be in standard or scientific notation. Use an `F` or `f` suffix for `float` values.
- **Character Literals**: Represent a single character enclosed in single quotes, including escape sequences and Unicode representations.
- **String Literals:** Represent sequences of characters in double quotes and are `String` objects stored in the string pool.
- **Boolean Literals**: Represent logical values with `true` and `false`.
- **Null Literal:** Represents the absence of a value and can be assigned to any reference type variable.

**Here is an example:**

```
int age = 25; // 25 is an integer literal

double pi = 3.14159; // 3.14159 is a floating-point literal

char grade = 'A'; // 'A' is a character literal

boolean isActive = true; // true is a boolean literal

String message = "Hello, Java!"; // "Hello, Java!" is a string literal
```

## 3. Operators

Java operators are special symbols that perform operations on variables and values. They are used to manipulate data and variables in expressions. Java provides a wide array of operators categorized into several types based on their functionality.

## Types of Java Operators

- **Arithmetic Operators**
- **Relational Operators**
- **Logical Operators**
- **Assignment Operators**
- **Unary Operators**
- **Bitwise Operators**
- **Ternary Operator**

## Arithmetic Operators

Arithmetic operators are used to perform basic mathematical operations.

- + (Addition)
- - (Subtraction)
- * (Multiplication)
- / (Division)
- % (Modulus)

## Example

```
int a = 10;

int b = 5;

System.out.println("Addition: " + (a + b));     // Output: 15

System.out.println("Subtraction: " + (a - b));   // Output: 5

System.out.println("Multiplication: " + (a * b));// Output: 50

System.out.println("Division: " + (a / b));     // Output: 2

System.out.println("Modulus: " + (a % b));      // Output: 0
```

### Relational Operators

Relational operators are used to compare two values.

- == (Equal to)
- != (Not equal to)
- > (Greater than)
- < (Less than)
- >= (Greater than or equal to)
- <= (Less than or equal to)

### Example

```
int a = 10;

int b = 5;

System.out.println(a == b); // Output: false

System.out.println(a > b);  // Output: true

System.out.println(a < b);  // Output: false
```

### Logical Operators

Logical operators are used to combine multiple boolean expressions.

- && (Logical AND)
- || (Logical OR)
- ! (Logical NOT)

**Example**

```
boolean x = true;

boolean y = false;

System.out.println(x && y); // Output: false

System.out.println(x || y); // Output: true

System.out.println(!x);     // Output: false
```

Powered By

**Assignment Operators**

Assignment operators are used to assign values to variables.

- = (Simple assignment)
- += (Add and assign)
- -= (Subtract and assign)
- *= (Multiply and assign)
- /= (Divide and assign)
- %= (Modulus and assign)

**Example**

```
int a = 10;

a += 5;  // Equivalent to a = a + 5

System.out.println(a); // Output: 15
```

### Unary Operators

Unary operators operate on a single operand.

- + (Unary plus)
- - (Unary minus)
- ++ (Increment)
- -- (Decrement)

### Example

```
int a = 10;

System.out.println(++a); // Output: 11

System.out.println(--a); // Output: 10
```

### Bitwise Operators

Bitwise operators perform operations on bits.

- & (Bitwise AND)
- | (Bitwise OR)
- ^ (Bitwise XOR)
- ~ (Bitwise NOT)
- << (Left shift)
- >> (Right shift)
- >>> (Unsigned right shift)

### Example

```
int a = 5;  // Binary: 0101
```

```
int b = 3;  // Binary: 0011
System.out.println(a & b); // Output: 1 (Binary: 0001)
```

### Ternary Operator

The ternary operator is a shorthand form of the if-else statement.

- ? : (Ternary)

### Example

```
int a = 10;
int b = 5;
int max = (a > b) ? a : b;
System.out.println(max); // Output: 10
```

## 4. operator precedence and associativity

The following table lists operators in order of highest precedence (top) to lowest (bottom), with their associativity:

| Precedence | Operator(s) | Type | Associativity |
|---|---|---|---|
| 15 | ( ), [ ], . | Parentheses, Array subscript, Member selection | Left to Right |

| | | | |
|---|---|---|---|
| 14 | `++`, `--` | Post-increment/decrement | Left to Right |
| 13 | `++`, `--`, `+`, `-`, `!`, `~`, `(type)` | Unary pre-increment/decrement, Unary plus/minus, Logical NOT, Bitwise complement, Cast | Right to Left |
| 12 | `*`, `/`, `%` | Multiplicative | Left to Right |
| 11 | `+`, `-` | Additive | Left to Right |
| 10 | `<<`, `>>`, `>>>` | Bitwise shift | Left to Right |
| 9 | `<`, `<=`, `>`, `>=`, `instanceof` | Relational | Left to Right |
| 8 | `==`, `!=` | Equality | Left to Right |
| 7 | `&` | Bitwise AND | Left to Right |
| 6 | `^` | Bitwise XOR (exclusive OR) | Left to Right |
| 5 | ` | ` | Bitwise OR |

| | | | |
|---|---|---|---|
| | | | (inclusive OR) |
| 4 | && | Logical AND | Left to Right |
| 3 | ` | | ` |
| 2 | ? : | Ternary conditional | Right to Left |
| 1 | =, +=, -=, *=, /=, etc. | Assignment | Right to Left |

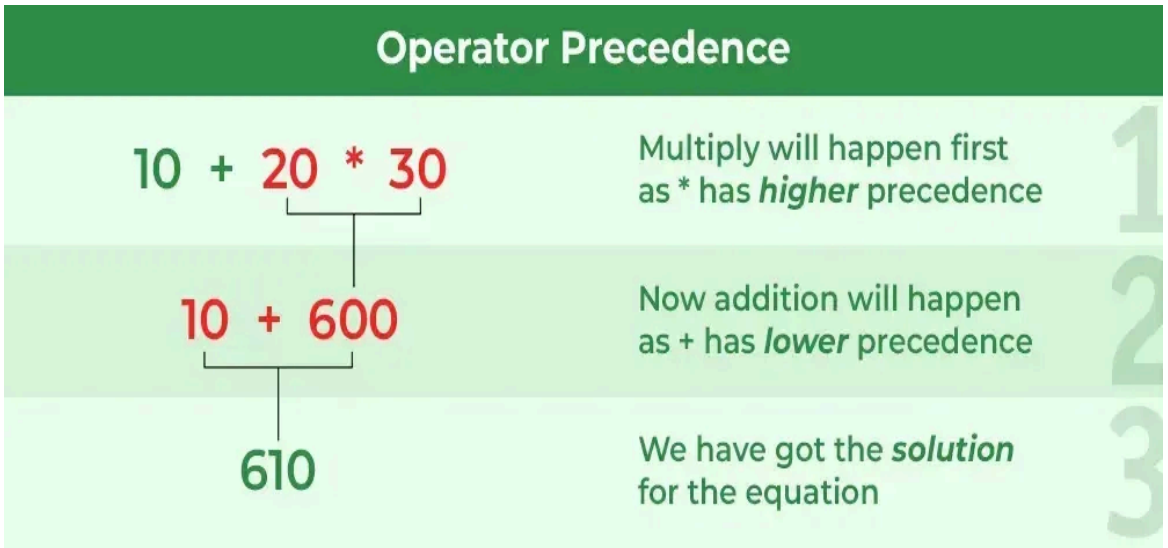
## Operator Precedence in Java

- In Java, operator precedence specifies the order in which operations are performed within an expression.
- When an expression contains multiple operators, those with higher precedence are evaluated before those with lower precedence.

**For expression:**

*10+20*30*

The expression contains two operators, + (addition) and * (multiplication). According to operator precedence, multiplication (*) has higher precedence than addition (+), so multiplication is checked first. After evaluating multiplication, the addition operator is then evaluated to give the final result.




## Operator Associativity in Java

Operator associativity in Java determines the order of evaluation when multiple operators of the same precedence appear in an expression.

**Java supports two types of associativity:**

**1. Left-to-Right Associativity**

When multiple operators of the same precedence appear in an expression, they are evaluated from left to right. For example, in the expression a + b - c, addition and subtraction have the same precedence and are left-associative, so the expression is evaluated as (a + b) - c.

**Operators with left-to-right associativity include:**

- Arithmetic operators: +, -, *, /, %
- Relational operators: >, <, >=, <=
- Logical AND/OR: &&, ||
- Bitwise operators: &, |, ^, <<, >>

**2. Right-to-Left Associativity**

Right-to-Left Associativity means that operators are evaluated from right to left when they have the same precedence.

**Operators with right-to-left associativity include:**

- Assignment operators: =, +=, -=, etc.
- Unary operators: ++, --, !, ~